

Documentazione Test

Ecco le sezioni formattate esattamente nello stile del tuo documento da aggiungere alla sezione **2. Test di Integrazione**.

1. Test di Unità

Test eseguiti in isolamento, senza accesso a file system, database o socket di rete.

Package: data

Classe Testata: ContinuousAttribute

- **Test 1 (testGetName):**
 - **Cosa testa:** La corretta ereditarietà e restituzione del nome simbolico dell'attributo numerico.
 - **Descrizione:** Inizializzando un attributo continuo con etichetta "chilometri", il test verifica che il metodo ereditato getName() restituisca esattamente la stringa attesa.
- **Test 2 (testGetIndex):**
 - **Cosa testa:** La corretta memorizzazione e restituzione dell'indice identificativo dell'attributo.
 - **Descrizione:** Fornendo l'indice numerico 2 al costruttore, si assicura che il metodo getIndex() restituisca il valore intero corrispondente.

Classe Testata: DiscreteAttribute

- **Test 1 (testNameAndIndex):**
 - **Cosa testa:** L'inizializzazione corretta del nome e dell'indice numerico ereditati dalla superclasse.
 - **Descrizione:** Verifica che i metodi getName() e getIndex() restituiscano i valori passati nel costruttore.
- **Test 2 (testNumberOfDistinctValues):**
 - **Cosa testa:** La cardinalità dell'insieme dei valori discreti gestiti dall'attributo.
 - **Descrizione:** Controlla che getNumberOfDistinctValue() restituisca il numero esatto di elementi inseriti.
- **Test 3 (testEliminazioneValoriDuplicati):**
 - **Cosa testa:** La proprietà intrinseca della struttura dati TreeSet nell'eliminazione dei duplicati.
 - **Descrizione:** Passando un array contenente valori ripetuti (es. "Fiat", "Fiat", "Ford"), assicura che il numero finale di valori distinti non consideri i doppi.
- **Test 4 (testIteratorReturnsValues):**
 - **Cosa testa:** L'implementazione dell'interfaccia Iterable .
 - **Descrizione:** Esegue un ciclo for-each per confermare che l'iteratore attraversi correttamente tutti i valori memorizzati.

Classe Testata: Attribute

- **Test 1 (testGetName):**
 - **Cosa testa:** La corretta memorizzazione e restituzione del nome simbolico dell'attributo.
 - **Descrizione:** Inizializzando l'attributo con l'etichetta "temperatura", il test verifica che il metodo getName() restituisca esattamente la stringa attesa.
- **Test 2 (testGetIndex):**
 - **Cosa testa:** La corretta memorizzazione e restituzione dell'identificativo numerico posizionale dell'attributo.
 - **Descrizione:** Fornendo l'indice 5 al costruttore, si assicura che il metodo getIndex() restituisca il valore intero corrispondente.

Classe Testata: TrainingDataException

- **Test 1 (testExceptionMessage):**
 - **Cosa testa:** L'inizializzazione corretta del messaggio di errore all'interno della gerarchia delle eccezioni.
 - **Descrizione:** Istanza l'eccezione passando una stringa diagnostica di prova al costruttore. Verifica poi che l'invocazione del metodo ereditato getMessage() restituisca esattamente il testo fornito, garantendo la tracciabilità dell'errore.

Package: Database

Classe Testata: Column

- **Test 1 (testGetColumnName):**
 - **Cosa testa:** L'inizializzazione e l'accesso al nome della colonna.
 - **Descrizione:** Verifica che getColumnName() restituisca esattamente il valore assegnato tramite costruttore.
- **Test 2 (testIsNumberTrue e testIsNumberFalse):**
 - **Cosa testa:** Il riconoscimento del tipo di dato (numerico o testuale).
 - **Descrizione:** Accerta che il metodo isNumber() restituisca true per colonne di tipo "number" e false per colonne di tipo "string".
- **Test 3 (testUnknownTypesNotNumber):**
 - **Cosa testa:** La robustezza del controllo numerico a fronte di tipi non convenzionali.
 - **Descrizione:** Fornendo un tipo generico (es. "date"), verifica che isNumber() non produca falsi positivi.
- **Test 4 (testToString):**
 - **Cosa testa:** La formattazione della rappresentazione testuale.
 - **Descrizione:** Valuta che il metodo toString() restituisca la stringa formattata secondo lo schema nome:tipo .

Classe Testata: Example

- **Test 1 (testAddAndGet):**
 - **Cosa testa:** L'ordine di inserimento e l'estrazione posizionale degli elementi della tupla.
 - **Descrizione:** Inserisce una stringa e un valore a virgola mobile, verificando che il metodo get(int) restituisca ciascun elemento all'indice corretto preservandone il tipo originario.
- **Test 2 (testGetOutOfBounds):**
 - **Cosa testa:** Il rispetto dei limiti della lista interna.
 - **Descrizione:** Richiede un indice non presente nel vettore e accerta che venga sollevata l'eccezione IndexOutOfBoundsException .
- **Test 3 (testToString):**
 - **Cosa testa:** La serializzazione testuale della tupla.
 - **Descrizione:** Verifica che l'invocazione di toString() produca una sequenza di elementi separati da spazio contenente tutti i valori inseriti.
- **Test 4 (testCompareToEqual e testCompareToDifferent):**
 - **Cosa testa:** Il comportamento di ordinamento definito dall'interfaccia Comparable .
 - **Descrizione:** Valuta che il confronto tra due tuple identiche restituisca 0 e che la presenza di un valore differente in una posizione produca un risultato diverso da zero.
- **Test 5 (testIterator):**
 - **Cosa testa:** L'iterazione tramite Iterable .
 - **Descrizione:** Attraversa la classe con un ciclo for-each per verificare che tutti gli elementi registrati vengano scorsi correttamente.

Classe Testata: DatabaseConnectionException

- **Test 1 (testMessage):**
 - **Cosa testa:** L'inizializzazione e la restituzione del messaggio diagnostico di errore.
 - **Descrizione:** Istanza l'eccezione con un messaggio descrittivo e verifica che getMessage() restituisca esattamente la stringa fornita.
- **Test 2 (testIsException):**
 - **Cosa testa:** La corretta collocazione nella gerarchia delle eccezioni Java.
 - **Descrizione:** Verifica tramite asserzione di tipo che l'eccezione estenda la classe standard java.lang.Exception , qualificandosi come checked exception.

Classe Testata: EmptySetException

- **Test 1 (testMessage):**
 - **Cosa testa:** L'inizializzazione e la restituzione del messaggio diagnostico di errore.

- **Descrizione:** Istanza l'eccezione senza argomenti e verifica che getMessage() restituisca esattamente la stringa standard [!] The result set is empty .
- **Test 2 (testIsException):**
 - **Cosa testa:** La corretta collocazione nella gerarchia delle eccezioni Java.
 - **Descrizione:** Accerta tramite asserzione di tipo che la classe erediti da java.lang.Exception , garantendone il comportamento come checked exception.

Package: Server

Classe Testata: UnknownValueException

- **Test 1 (testMessage):**
 - **Cosa testa:** La corretta inizializzazione e conservazione del messaggio di errore diagnostico.
 - **Descrizione:** Istanza l'eccezione passando una stringa specifica al costruttore e verifica che l'invocazione di getMessage() restituisca esattamente il testo fornito originariamente.
- **Test 2 (testIsException):**
 - **Cosa testa:** La corretta posizione della classe all'interno della gerarchia delle eccezioni standard.
 - **Descrizione:** Verifica tramite un'asserzione di tipo che la classe estenda direttamente java.lang.Exception , qualificandosi come checked exception che obbliga i chiamanti a gestirne esplicitamente la cattura.

2. Test di Integrazione

Test che verificano l'interazione tra i moduli interni o con sistemi esterni (es. database MySQL).

Package: tree

Classe Testata: DiscreteNode

- **Test 1 (testGenerazioneRami):**
 - **Cosa testa:** Il corretto popolamento della lista dei nodi figli in base ai valori distinti dell'attributo.
 - **Descrizione:** Verifica che, inizializzando il nodo con i dati della colonna discreta X (avente valori "A" e "B"), vengano generati esattamente 2 rami distinti per il partizionamento.
- **Test 2 (testCondizioneValoreEsistente):**
 - **Cosa testa:** La corretta associazione tra un valore di input noto e il rispettivo ramo di split.
 - **Descrizione:** Passando alla funzione testCondition il valore esistente "A", il test si assicura che il metodo restituisca un indice valido (0 o 1) corrispondente al ramo

generato.

■ **Test 3 (testCondizioneValoreInesistente):**

- **Cosa testa:** La gestione di valori non previsti nel training set originale.
- **Descrizione:** Fornendo in input un valore testuale non mappato ("Z"), il test verifica che la funzione testCondition restituisca rigorosamente -1 .

Classe Testata: ContinuousNode

■ **Test 1 (testGenerazioneRami):**

- **Cosa testa:** La creazione dei rami di split basata sull'algoritmo di minimizzazione della varianza per attributi continui.
- **Descrizione:** Utilizzando i dati numerici della colonna Y , si verifica che l'algoritmo identifichi una soglia di split valida, generando di conseguenza almeno un ramo figlio (tipicamente due, per i valori $\leq$ e $>$).

■ **Test 2 (testValutazioneSoglia):**

- **Cosa testa:** Il corretto instradamento di un valore continuo attraverso gli operatori relazionali del nodo.
- **Descrizione:** Sottoponendo a testCondition un input numerico (es. 1.0), il test si assicura che l'indice restituito identifichi coerentemente il ramo di partizione corretto oppure l'assenza di corrispondenza (-1).

Classe Testata: LeafNode

■ **Test 1 (testCalcoloValorePredetto):**

- **Cosa testa:** La correttezza aritmetica del calcolo della media dei valori target.
- **Descrizione:** Assicura che il valore predetto calcolato nel costruttore per gli esempi assegnati alla foglia sia un numero in virgola mobile formalmente valido (non NaN).

■ **Test 2 (testNumeroRamiFigli):**

- **Cosa testa:** L'invariante strutturale primaria dei nodi terminali dell'albero.
- **Descrizione:** Verifica esplicitamente che il metodo getNumberOfChildren() restituisca 0 , confermando che la foglia non possiede alcun ramo discendente.

■ **Test 3 (testToString):**

- **Cosa testa:** La formattazione della rappresentazione testuale del nodo terminale.
- **Descrizione:** Controlla che la stringa generata dal metodo toString() contenga esplicitamente la parola chiave "LEAF" , fondamentale per la corretta visualizzazione delle regole dell'albero.

Classe Testata: Node (testata tramite l'estensione concreta LeafNode)

■ **Test 1 (testInizializzazioneIndici):**

- **Cosa testa:** L'assegnamento degli estremi dell'intervallo degli esempi.

- **Descrizione:** Verifica che i metodi getter restituiscano esattamente gli indici inizio e fine passati al costruttore del nodo.
- **Test 2 (testGenerazioneIdProgressivo):**
 - **Cosa testa:** L'incremento della variabile statica contatore per l'ID del nodo.
 - **Descrizione:** Crea un secondo nodo e verifica che il suo idNode sia strettamente maggiore di quello del primo nodo generato nel setup.
- **Test 3 (testCalcoloVarianza):**
 - **Cosa testa:** Il risultato del calcolo della somma dei quadrati degli scarti (SSE).
 - **Descrizione:** Assicura che la varianza calcolata nel costruttore sia matematicamente coerente, ovvero non negativa (≥ 0.0).
- **Test 4 (testToString):**
 - **Cosa testa:** Il contenuto della stringa descrittiva dell'oggetto.
 - **Descrizione:** Controlla che il metodo restituisca una stringa contenente i riferimenti chiave agli indici coperti e al valore della varianza.

Classe Testata: SplitNode (testata mediante l'implementazione concreta DiscreteNode)

- **Test 1 (testGetAttribute):**
 - **Cosa testa:** L'incapsulamento dell'attributo indipendente su cui è generato lo split.
 - **Descrizione:** Verifica che l'invocazione di getAttribute() restituisca esattamente l'istanza dell'oggetto Attribute passata al momento della costruzione del nodo.
- **Test 2 (testCalcoloSplitVariance):**
 - **Cosa testa:** La validità matematica della varianza di split calcolata all'interno del costruttore.
 - **Descrizione:** assicura che il calcolo della splitVariance (ottenuta sommando le varianze di ciascun ramo figlio individuato nel sotto-insieme di training) generi un valore reale non negativo.
- **Test 3 (testGestioneFigliESplitInfo):**
 - **Cosa testa:** Il corretto popolamento della struttura dati dei rami figli (mapSplit) e l'accesso agli oggetti aggregati SplitInfo .
 - **Descrizione:** Controlla che getNumberOfChildren() confermi la presenza di rami e che il metodo getSplitInfo(int) restituisca un'istanza valida e non nulla per il primo indice richiesto.
- **Test 4 (testFormulateQuery):**
 - **Cosa testa:** L'esportazione testuale delle condizioni di test (regole di split).
 - **Descrizione:** Verifica che il metodo iteri sui rami figli concatenando le relative stringhe in un'unica query formattata, accertandosi che essa includa il nome dell'attributo di split.

■ Test 5 (testCompareTo):

- **Cosa testa:** L'implementazione dell'interfaccia Comparable .
- **Descrizione:** Istanza un secondo nodo con caratteristiche e dati analoghi, invocando compareTo() per assicurarsi che il confronto basato sulle varianze equivalenti restituisca regolarmente il valore 0 .

Classe Testata: RegressionTree

■ Test 1 (testInduzioneAlbero):

- **Cosa testa:** La corretta costruzione strutturale dell'albero a partire dal dataset.
- **Descrizione:** Inizializzando la classe con i dati della tabella provaC , il test verifica che il costruttore istanzi correttamente la gerarchia dei nodi. Si accerta che la rappresentazione testuale dell'albero generato non sia nulla e contenga i nodi previsti ("SPLIT" o "LEAF").

■ Test 2 (testComportamentoNodoRadice):

- **Cosa testa:** Il comportamento dei metodi di estrazione dati sul nodo radice.
- **Descrizione:** Poiché la radice dell'albero basato su provaC è un nodo decisionale e non terminale, il test verifica che getCurrentNodeQuery() restituisca una stringa valida (la condizione di split) e che getPredictedValue() restituisca null (comportamento atteso per un nodo che non è una foglia).

■ Test 3 (testNavigazioneEccezione):

- **Cosa testa:** La corretta gestione degli errori quando si richiede un ramo figlio inesistente.
- **Descrizione:** Si forza il passaggio di un indice non valido (es. -1) al metodo getChild() , verificando che la classe intercetti l'errore e sollevi correttamente l'eccezione personalizzata UnknownValueException .

■ Test 4 (testSerializzazione):

- **Cosa testa:** Il salvataggio e il caricamento dell'albero su file tramite l'interfaccia Serializable .
- **Descrizione:** L'istanza dell'albero viene salvata in un file temporaneo locale per verificarne l'effettiva creazione. Successivamente, il file viene ricaricato in memoria e si verifica che la struttura del nuovo oggetto sia identica a quella originale (confrontando i rispettivi output di toString()).

Package: data

Classe Testata: Data

■ Test 1 (tabellaInesistente_throwsTrainingDataException):

- **Cosa testa:** La gestione degli errori di inizializzazione legati al database.
- **Descrizione:** Invocando il costruttore con il nome di una tabella non esistente, verifica che il sistema sollevi l'eccezione personalizzata TrainingDataException

anziché generare un errore di sistema non gestito.

■ **Test 2 (tabellaConSingolaColonna_throwsTrainingDataException):**

- **Cosa testa:** La validazione strutturale dello schema relazionale della tabella.
- **Descrizione:** Assicura che, fornendo una tabella con meno di due colonne (priva del requisito minimo di almeno un attributo esplicativo e uno target), venga regolarmente sollevata l'eccezione TrainingDataException .

■ **Test 3 (targetNonNumerico_throwsTrainingDataException):**

- **Cosa testa:** Il vincolo di dominio sull'attributo di classe.
- **Descrizione:** Verifica che l'algoritmo respinga (tramite eccezione) dataset la cui ultima colonna (il target da predire) non sia di tipo numerico continuo, come strettamente richiesto per gli alberi di regressione.

■ **Test 4 (identificaTipoColonnaCorrettamente):**

- **Cosa testa:** Il meccanismo dinamico di mappatura dei tipi sulle colonne della tabella.
- **Descrizione:** Interroga il dataset appena caricato (provaC) e asserisce che la colonna testuale X venga correttamente identificata come DiscreteAttribute e la colonna numerica Y come ContinuousAttribute .

■ **Test 5 (getExplanatoryValue_ritornaValoreNonNullo e _throwsIndexOutOfBoundsException):**

- **Cosa testa:** Le garanzie di affidabilità e i limiti di accesso alla matrice di esempi.
- **Descrizione:** Il primo collaudo verifica l'effettiva estrazione di un oggetto non nullo prelevato da indici validi, mentre il secondo forza un accesso fuori range verificando l'intercettazione dell'errore tramite IndexOutOfBoundsException .

■ **Test 6 (getClassAttribute_attributoTargetCorretto e getClassValue_ritornaIndiceValido):**

- **Cosa testa:** Il corretto binding e la lettura della variabile target.
- **Descrizione:** Verifica che getClassAttribute() punti all'ultima colonna del set (identificata dal nome C e dal relativo indice) e che getClassValue() riesca a estrarre un valore numerico in formato Double senza causare errori.

■ **Test 7 (ordinamentoCorretto_discreteAttribute):**

- **Cosa testa:** L'implementazione dell'algoritmo di ordinamento interno (Quicksort) sui valori testuali.
- **Descrizione:** Dopo aver invocato data.sort() sull'attributo discreto X , il test itera sulle righe accertandosi che la post-condizione dell'ordinamento alfabetico (senso crescente) sia rigorosamente rispettata.

■ **Test 8 (ordinamentoCorretto_continuousAttribute):**

- **Cosa testa:** L'implementazione dell'algoritmo di ordinamento interno (Quicksort) sui valori reali.
- **Descrizione:** Invoca data.sort() sull'attributo continuo Y e verifica ciclicamente che l'ordinamento numerico in senso crescente sia stato correttamente applicato a tutti gli elementi della colonna.

Package: Database

Classe Testata: TableData

▪ Test 1 (testGetTransazioniValide):

- **Cosa testa:** L'esecuzione della query SELECT e il mappaggio dei ResultSet in una lista ordinata di tuple Example .
- **Descrizione:** Invoca l'estrazione sulla tabella provaC e verifica che la struttura dati restituita non sia nulla né vuota.

▪ Test 2 (testGetTransazioniTabellaInesistente):

- **Cosa testa:** La gestione degli errori SQL per richieste malformate.
- **Descrizione:** Tenta di interrogare una tabella inesistente per accertarsi che il metodo intercetti il problema sollevando la corretta SQLException .

▪ Test 3 (testGetDistinctColumnValuesString):

- **Cosa testa:** L'estrazione e l'ordinamento (tramite TreeSet) dei valori unici di tipo String .
- **Descrizione:** Esegue una query SELECT DISTINCT sulla colonna testuale X e valuta che l'insieme popolato non sia vuoto.

▪ Test 4 (testGetDistinctColumnValuesNumber):

- **Cosa testa:** L'estrazione e l'ordinamento (tramite TreeSet) dei valori unici di tipo Double .
- **Descrizione:** Esegue una query SELECT DISTINCT sulla colonna numerica Y e verifica che il framework di persistenza gestisca correttamente i valori reali.

Classe Testata: TableSchema

▪ Test 1 (testGetNumberOfAttributes):

- **Cosa testa:** L'estrazione e il conteggio delle colonne dal dizionario dei dati del database.
- **Descrizione:** Accerta che il metodo getNumberOfAttributes() restituisca il numero corretto di attributi, verificando che il valore sia strettamente maggiore di zero per una tabella esistente.

▪ Test 2 (testGetColumn e testGetColumnOutOfBounds):

- **Cosa testa:** L'accesso posizionale alla definizione dei tipi della singola colonna.

- **Descrizione:** Verifica che getColumn(int) restituisca l'oggetto Column corrispondente all'indice fornito e garantisce il sollevamento dell'eccezione IndexOutOfBoundsException per richieste relative a posizioni inesistenti.
- **Test 3 (testIterator):**
 - **Cosa testa:** Iterazione garantita dall'implementazione dell'interfaccia Iterable .
 - **Descrizione:** Esegue un costrutto iterativo for-each per scorrere sequenzialmente le colonne dello schema, accertando che il numero di cicli corrisponda esattamente alla cardinalità degli attributi.
- **Test 4 (testTabellaInesistente):**
 - **Cosa testa:** Il comportamento del costruttore in risposta alla scansione dei metadati per una tabella non definita nel database.
 - **Descrizione:** Fornendo il nome di una tabella fittizia, assicura che il sistema non vada in crash ma si limiti a esaurire regolarmente il ResultSet , mantenendo la lista interna vuota (cardinalità pari a 0).

Classe Testata: DbAccess

- **Test 1 (testInitAndGetConnection):**
 - **Cosa testa:** L'inizializzazione della connessione al database MySQL e il recupero della sessione attiva tramite getConnection() .
 - **Descrizione:** Invoca initConnection() per caricare il driver JDBC e stabilire il collegamento con i parametri di rete predefiniti. Accerta che l'oggetto restituito non sia nullo e che la connessione risulti attivamente aperta, assicurandosi che non vengano sollevate eccezioni del tipo DatabaseConnectionException .
- **Test 2 (testCloseConnection):**
 - **Cosa testa:** La chiusura e il rilascio appropriato delle risorse di rete allocate per la sessione.
 - **Descrizione:** Avvia la connessione e successivamente invoca il metodo closeConnection() , interrogando l'oggetto di connessione sottostante per verificare che il suo stato sia formalmente chiuso, evitando così memory leak o connessioni pendenti sul DBMS.
- **Test 3 (testGetAvailableTablesSuccess):**
 - **Cosa testa:** L'estrazione dell'elenco delle tabelle utente tramite i metadati del database relazionale.
 - **Descrizione:** Inizializza la connessione al database e invoca il metodo `getAvailableTables()`. Verifica che la lista restituita non sia nulla, che contenga almeno un elemento e che tra le tabelle rilevate sia presente la tabella di prova `provac`. Accerta inoltre che vengano filtrate unicamente le entità di tipo `TABLE`, escludendo viste e tabelle di sistema.

- **Test 4 (testGetAvailableTablesConnectionClosed):**

- **Cosa testa:** La corretta propagazione degli errori SQL in caso di sessione non attiva.
- **Descrizione:** Chiude preventivamente la connessione con `closeConnection()` e successivamente tenta di invocare `getAvailableTables()`. Accerta che il metodo intercetti l'impossibilità di raggiungere il DBMS sollevando formalmente l'eccezione `SQLException`, evitando ritorni incoerenti o crash non gestiti.

Package: Server

Classe Testata: MultiServer

- **Test 1 (testServerAvvioEConnessione):**
 - **Cosa testa:** L'avvio del servizio di rete e la corretta accettazione delle richieste in ingresso.
 - **Descrizione:** Avvia il server su una porta di test sfruttando un thread secondario per non bloccare l'esecuzione. Successivamente, crea un Socket client e tenta la connessione, verificando tramite l'asserzione `assertTrue` che il collegamento venga stabilito correttamente e che il server sia quindi regolarmente in ascolto.

Classe Testata: ServerOneClient

- **Test 1 (testAcquisizioneDatiOK):**
 - **Cosa testa:** L'elaborazione del comando 0 per l'acquisizione dei dati e la corretta istanziazione di `ServerOneClient` come thread dedicato.
 - **Descrizione:** Avvia un `ServerSocket` temporaneo e istanzia `ServerOneClient` passando il socket connesso. Il test agisce da client inviando l'intero 0 e la stringa "provaC" tramite un `ObjectOutputStream`. Intercetta quindi la stringa in risposta, verificando che corrisponda esattamente a "OK".
- **Test 2 (testCaricamentoAlberoFallito):**
 - **Cosa testa:** L'elaborazione del comando 2 e la gestione delle eccezioni per il caricamento di file .dmp mancanti.
 - **Descrizione:** Invia il comando intero 2 seguito dal nome di una tabella non serializzata in precedenza. Si assicura che il blocco catch interno venga innescato regolarmente, restituendo la stringa di errore cablata "Errore durante il caricamento da archivio."
- **Test 3 (testRecuperoTabelleOK):**
 - **Cosa testa:** L'elaborazione del comando 4 per il recupero asincrono delle tabelle disponibili nel database e la serializzazione degli oggetti di risposta.
 - **Descrizione:** Tramite uno stream di output su socket di test, invia il comando intero 4 a `ServerOneClient`. Verifica che la prima risposta ricevuta sia la stringa di sincronizzazione "OK" e che l'oggetto successivo sia un'istanza di `List<String>`. Controlla infine che la lista non sia vuota e includa la tabella di test `provac`.
- **Test 4 (testRecuperoTabelleFallito):**

- **Cosa testa:** La resilienza del thread di servizio e il protocollo di notifica d'errore a fronte di anomalie nel livello di persistenza durante l'esecuzione del comando 4.
- **Descrizione:** Simula l'invio del comando intero 4 forzando una condizione di errore a livello database (es. connessione DBMS non disponibile). Verifica che `ServerOneClient` non interrompa improvvisamente il thread ma trasmetta al client il token di fallimento (es. "ERROR" o messaggio diagnostico di eccezione), preservando l'integrità del canale socket per consentire al client di intercettare il problema.